

FRENCH CHEESE SHOP

AI in Requirements Engineering

A French cheese shop demo asks: what does the user mean, what context is needed, and what makes the answer useful?

Juho Vepsäläinen

6.5.2026

**Ambiguity
Data
Evaluation**



Photo: jackmac34 / Pixabay

SESSION FOCUS

Learning Outcomes

1

Interpret the request

Identify what the request leaves unspecified.

SESSION FOCUS

Learning Outcomes

1

Interpret the request

Identify what the request leaves unspecified.

2

Specify the context

Name the data and shop knowledge the system needs.

SESSION FOCUS

Learning Outcomes

1

Interpret the request

Identify what the request leaves unspecified.

2

Specify the context

Name the data and shop knowledge the system needs.

3

Evaluate the response

Judge whether the answer is useful under ambiguity.

CUSTOMER QUERY

A Vague Customer Request



“I want something
like Brie,
but stronger.”

Easy for a shopkeeper who knows the cheeses
and the customer. Hard for a system without both.

Photo: lee_2 / Pixabay

CUSTOMER QUERY

A Vague Customer Request



“I want something
like Brie,
but stronger.”

Easy for a shopkeeper who knows the cheeses
and the customer. Hard for a system without both.

Photo: lee_2 / Pixabay

Similar how?

Texture, milk type,
region, serving context

CUSTOMER QUERY

A Vague Customer Request



“I want something
like Brie,
but stronger.”

Easy for a shopkeeper who knows the cheeses
and the customer. Hard for a system without both.

Photo: lee_2 / Pixabay

Similar how?

Texture, milk type,
region, serving context

Stronger how?

Age, aroma,
pungency, salt

CUSTOMER QUERY

A Vague Customer Request



“I want something
like Brie,
but stronger.”

Easy for a shopkeeper who knows the cheeses
and the customer. Hard for a system without both.

Photo: lee_2 / Pixabay

Similar how?

Texture, milk type,
region, serving context

Stronger how?

Age, aroma,
pungency, salt

What limits apply?

Budget, stock,
allergies, pairings

CUSTOMER QUERY

A Vague Customer Request



“I want something
like Brie,
but stronger.”

Easy for a shopkeeper who knows the cheeses
and the customer. Hard for a system without both.

Photo: lee_2 / Pixabay

Similar how?

Texture, milk type,
region, serving context

Stronger how?

Age, aroma,
pungency, salt

What limits apply?

Budget, stock,
allergies, pairings

What is useful?

Recommend, ask,
or explain the tradeoff

CONCEPTUAL SHIFT

Traditional RE vs AI-Supported RE

The key shift is from specifying functions to defining behavior under uncertainty.

Traditional

- Specify search and filter behavior
- Validate fields and stock
- Define expected outputs
- Success = feature behaves as specified

CONCEPTUAL SHIFT

Traditional RE vs AI-Supported RE

The key shift is from specifying functions to defining behavior under uncertainty.

Traditional

- Specify search and filter behavior
- Validate fields and stock
- Define expected outputs
- Success = feature behaves as specified

AI-supported

- Interpret vague language
- Use data and domain knowledge
- Explain uncertain recommendations
- Success = useful, defensible fit

CONCEPTUAL SHIFT

Traditional RE vs AI-Supported RE

The key shift is from specifying functions to defining behavior under uncertainty.

Traditional

- Specify search and filter behavior
- Validate fields and stock
- Define expected outputs
- Success = feature behaves as specified

AI-supported

- Interpret vague language
- Use data and domain knowledge
- Explain uncertain recommendations
- Success = useful, defensible fit

Requirements go beyond functions to specify how the system interprets vague requests.

Live Demo

Watch what the system infers, what shop data it uses, and what checks make the answer defensible.

Open the demo

french-cheese-shop-
demo.survivejs.workers.dev

Scan the QR code



The demo generalizes: when systems interpret,
requirements must cover uncertainty.

That means explicit needs, domain data, and visible evaluation checks.

CLOSING

Three Takeaways

Interpretation

Turn hidden preferences into explicit requirements.

CLOSING

Three Takeaways

Interpretation

Turn hidden preferences into explicit requirements.

Context

Use product data and shop constraints, not wording alone.

CLOSING

Three Takeaways

Interpretation

Turn hidden preferences into explicit requirements.

Context

Use product data and shop constraints, not wording alone.

Evaluation

Make success criteria visible as checks.

Questions and Comments

References

Cited sources

[1]

B. Nuseibeh and S. Easterbrook. "Requirements Engineering: A Roadmap." In The Future of Software Engineering, ICSE 2000.

[2]

N. Kiyavitskaya, N. Zeni, L. Mich, and D. M. Berry. "Requirements for tools for ambiguity identification and measurement in natural language requirements specifications." Requirements Engineering, 13(3), 2008.

[3]

K. Ahmad, M. Abdelrazek, C. Arora, M. Bano, and J. Grundy. "Requirements engineering for artificial intelligence systems: A systematic mapping study." Information and Software Technology, 158, 2023.

[4]

P. Lipsanen, L. Rannikko, F. Christophe, K. Kalliokoski, V. Stirbu, and T. Mikkonen. "Shift-Up: A Framework for Software Engineering Guardrails in AI-native Software Development." arXiv:2604.20436, 2026.

[5]

H. Cheng, J. H. Husen, Y. Lu, T. Racharak, N. Yoshioka, N. Ubayashi, and H. Washizaki. "Generative AI for Requirements Engineering: A Systematic Literature Review." Software: Practice and Experience, 56(2), 2026.

[6]

A. Ferrari and P. Spoletini. "Formal requirements engineering and large language models: A two-way roadmap." Information and Software Technology, 181, 2025.